

DOCUMENTACIÓN TÉCNICA DEL PROYECTO

JUEGO MENTAL MATEMÁTICO

Introducción

El proyecto "Juego Mental Matemático" es una aplicación desarrollada en Java utilizando JavaFX y el patrón de arquitectura MVC (Modelo – Vista – Controlador). El objetivo principal del sistema es permitir que los jugadores identifiquen reglas matemáticas a través de diferentes niveles y dinámicas de juego.

El sistema se encuentra estructurado en:

- Modelo: Maneja la lógica del negocio y las entidades principales.
- Vista: Interfaces gráficas construidas en JavaFX y archivos FXML.
- Controladores: Gestionan la interacción entre la vista y el modelo.
- Servicios: Manejan la persistencia de datos y conexión con base de datos.

La aplicación cuenta con funcionalidades como:

- Registro y continuación de jugadores.
- Sistema de niveles.
- Validación de respuestas.
- Persistencia de puntajes.
- Integración con base de datos.
- Interfaz gráfica moderna.
- Sistema de navegación entre ventanas.

ARQUITECTURA DEL PROYECTO

El sistema utiliza el patrón MVC:

Modelo

Contiene las clases encargadas de la lógica del juego.

Clases principales:

- Juego

- Jugador
- Nivel
- Regla
- TablaIO

Vista

Contiene los archivos FXML y componentes gráficos.

Interfaces principales:

- InterfazUsuarioFinal.fxml
- InterfazSelectorNivel.fxml
- InterfazAdivinarReglaJuego.fxml
- InterfazJuegoPrincipal.fxml
- InterfazJuegoGanado.fxml

Controladores

Gestionan eventos, navegación y comunicación entre interfaz y modelo.

Controladores principales:

- UsuarioController
- SelectorNivelController
- AdivinarReglaController
- JuegoPrincipalController
- JuegoGanadoController

Servicios

Gestionan conexión y persistencia de datos.

Servicios principales:

- ConexionBD
- DAOJugador
- DAOPartida

- CRUD

DOCUMENTACIÓN DE CONTROLADORES

1. UsuarioController

Descripción General

El controlador UsuarioController administra la pantalla inicial del sistema. Permite:

- Registrar un nuevo jugador.
- Continuar una partida existente.
- Validar nombres de usuario.
- Recuperar información almacenada en base de datos.
- Navegar hacia el selector de niveles.

Este controlador representa el punto de entrada principal de la aplicación.

Componentes FXML Utilizados

Componente	Tipo	Función
txtNombre	TextField	Captura el nombre del jugador
nameLabel	Label	Muestra el nombre escrito dinámicamente
lblMensaje	Label	Presenta mensajes de error o validación
lblPuntajeRecuperado	Label	Muestra el puntaje recuperado
panelPuntajeRecuperado	VBox	Panel visual de recuperación
btnJugar	Button	Inicia una nueva partida
btnContinuar	Button	Continúa una partida existente

Atributos Principales

juego

```
private final Juego juego = new Juego();
```

Representa la instancia principal del juego.

daoJugador

```
private final DAOJugador daoJugador = new DAOJugador();
```

Permite acceder a la persistencia de jugadores en la base de datos.

Métodos Principales

initialize()

Descripción

Método ejecutado automáticamente al cargar la interfaz.

Funcionalidad

- Escucha cambios en el campo de texto.
- Actualiza el nombre mostrado en pantalla.
- Oculta mensajes anteriores.
- Reinicia paneles visuales.

Objetivo

Mantener la interfaz dinámica y limpia visualmente.

onJugar()

Descripción

Permite iniciar una nueva partida.

Funcionalidad

1. Obtiene el nombre ingresado.
2. Valida que el nombre sea correcto.
3. Verifica si el usuario ya existe.
4. Si existe, muestra mensaje de error.

5. Si no existe:
 - crea el jugador
 - inicia el juego
 - guarda el jugador en BD
6. Navega hacia el selector de niveles.

Validaciones realizadas

- Nombre vacío.
- Usuario duplicado.
- Persistencia correcta.

Importancia

Evita duplicación de jugadores y garantiza integridad de datos.

onContinuar()

Descripción

Permite continuar una partida previamente guardada.

Funcionalidad

1. Busca el jugador en la base de datos.
2. Recupera su información.
3. Recupera puntaje y progreso.
4. Muestra información recuperada.
5. Navega hacia el selector de niveles.

Objetivo

Permitir persistencia y continuidad del juego.

validarNombre(String nombre)

Descripción

Valida que el nombre ingresado sea correcto.

Validaciones

- No vacío.
- Longitud adecuada.
- Formato permitido.

Retorno

- true → válido.
 - false → inválido.
-

mostrarMensajeError(String mensaje)

Descripción

Muestra mensajes de error visualmente.

Funcionalidad

- Cambia el texto del label.
 - Modifica estilos visuales.
 - Informa errores al usuario.
-

ocultarMensaje()

Descripción

Ocultar mensajes previos.

Objetivo

Mantener limpia la interfaz.

navegarA(String fxml, ControllerCallback callback)

Descripción

Gestiona la navegación entre ventanas.

Funcionalidad

- Carga archivos FXML.
- Obtiene controlador.
- Inyecta información.
- Cambia la escena actual.

Importancia

Centraliza el flujo de navegación del sistema.

2. SelectorNivelController

Descripción General

Este controlador administra la selección de niveles del juego.

Permite:

- Seleccionar niveles.
 - Inicializar partidas.
 - Abrir reglas del juego.
 - Navegar hacia la fase de deducción.
-

Componentes FXML

Componente	Tipo	Función
------------	------	---------

btn_level1	Button	Nivel 1
------------	--------	---------

btn_level2	Button	Nivel 2
------------	--------	---------

btn_level3	Button	Nivel 3
------------	--------	---------

btn_level4	Button	Nivel 4
------------	--------	---------

btn_level5	Button	Nivel 5
------------	--------	---------

btn_level6	Button	Nivel 6
------------	--------	---------

Componente	Tipo	Función
------------	------	---------

btn_level7	Button	Nivel 7
------------	--------	---------

Atributos

juego

private Juego juego;

Almacena la partida actual.

Métodos Principales

setJuego(Juego juego)

Descripción

Recibe la instancia del juego desde el controlador anterior.

Objetivo

Mantener persistencia del estado del juego.

irNivel1() hasta irNivel7()

Descripción

Métodos asociados a cada botón de nivel.

Funcionalidad

- Definen el índice del nivel.
- Llaman al método iniciarDesdeNivel().

Objetivo

Permitir navegación rápida hacia cualquier nivel.

iniciarDesdeNivel(int indice)

Descripción

Inicializa un nivel específico.

Funcionalidad

1. Define nivel actual.
2. Inicializa el nivel.
3. Navega hacia la fase de deducción.

Importancia

Representa el inicio oficial de la partida.

abrirReglas()

Descripción

Abre la ventana con las reglas del juego.

Funcionalidad

- Crea una nueva ventana modal.
- Carga la clase GameRulesWindow.
- Muestra reglas e instrucciones.

Objetivo

Ayudar al usuario a comprender el funcionamiento del juego.

navegarA(String fxml, ControllerCallback callback)

Descripción

Gestiona el cambio de ventanas.

Funcionalidad

- Carga FXML.
- Obtiene controlador.
- Inyecta dependencias.
- Actualiza la escena.

3. AdivinarReglaController

Descripción General

Controlador principal de la primera fase del juego.

En esta fase:

- El usuario ingresa números.
- El sistema devuelve resultados.
- El jugador debe deducir la regla matemática.

Cada intento incorrecto disminuye el puntaje.

Componentes FXML

Componente	Tipo	Función
txtEntrada	TextField	Entrada numérica
btnIngresar	Button	Procesar entrada
btnConozcaRegla	Button	Mostrar explicación
tablaVista	TableView	Tabla de entradas y salidas
colEntrada	TableColumn	Columna entrada
colSalida	TableColumn	Columna salida
lblPuntaje	Label	Puntaje actual
lblNivel	Label	Nivel actual

Clases Internas

FilaTabla

Descripción

Representa una fila de la tabla.

Atributos

- entrada
- salida

Objetivo

Mostrar dinámicamente resultados en pantalla.

Métodos Principales

initialize()

Descripción

Configura la tabla visual.

Funcionalidad

- Define columnas.
- Centra contenido.
- Configura estilos.
- Personaliza encabezados.

Importancia

Mejora la experiencia visual.

setJuego(Juego juego)

Descripción

Recibe el estado actual del juego.

Funcionalidad

- Guarda la referencia.
 - Obtiene nivel activo.
 - Actualiza información visual.
-

onIngresar()

Descripción

Procesa el número ingresado.

Flujo

1. Obtiene el número.
2. Valida formato.
3. Aplica regla matemática.
4. Genera resultado.
5. Agrega fila a la tabla.
6. Reduce puntaje.
7. Verifica derrota.

Validaciones

- Entrada vacía.
- Valores inválidos.
- Errores numéricos.

Objetivo

Permitir al jugador descubrir la regla.

actualizarPuntaje()

Descripción

Actualiza visualmente el puntaje.

Objetivo

Mantener sincronía entre modelo e interfaz.

verificarDerrota()

Descripción

Determina si el jugador perdió.

Condición

Puntaje igual a cero.

Resultado

Finaliza partida.

mostrarRegla()

Descripción

Permite mostrar la regla matemática.

Funcionalidad

- Obtiene la regla activa.
- La muestra visualmente.

Objetivo

Ayudar al jugador.

avanzarFase2()

Descripción

Envía al usuario a la segunda fase.

Objetivo

Continuar flujo del juego.

4. JuegoPrincipalController

Descripción General

Controlador encargado de la segunda fase del juego.

En esta fase:

- El jugador debe completar salidas correctas.

- Se valida la deducción de la regla.
- Se calculan respuestas correctas.
- Se determina si gana o pierde.

Componentes FXML

Componente	Tipo	Función
tabla	TableView	Tabla de respuestas
colEntrada	TableColumn	Entrada matemática
colSalida	TableColumn	Respuesta usuario
lblPuntaje	Label	Puntaje
lblNivel	Label	Nivel actual
btnTema	Button	Cambio de tema

Clase Interna

FilaFase2

Descripción

Representa cada fila editable de respuestas.

Atributos

- entrada
- salidaTexto
- salidaIngresada
- respuestaCorrecta

Objetivo

Permitir validación dinámica de respuestas.

Métodos Principales

initialize()

Descripción

Inicializa configuración de la tabla.

Funcionalidad

- Configura columnas.
 - Habilita edición.
 - Personaliza celdas.
 - Aplica estilos.
-

setJuego(Juego juego)

Descripción

Recibe la instancia del juego.

Funcionalidad

- Obtiene nivel actual.
 - Genera datos iniciales.
 - Actualiza interfaz.
-

generarFilas()

Descripción

Genera las entradas matemáticas.

Objetivo

Crear ejercicios dinámicos.

verificarRespuestas()

Descripción

Valida las respuestas del jugador.

Funcionalidad

1. Recorre todas las filas.
2. Evalúa resultados.
3. Marca correctas e incorrectas.
4. Calcula puntaje.
5. Determina victoria.

Importancia

Representa la validación principal del juego.

calcularResultadoCorrecto(int entrada)

Descripción

Aplica la regla matemática.

Objetivo

Obtener resultado esperado.

mostrarResultadoFinal()

Descripción

Muestra la pantalla final del nivel.

Funcionalidad

- Envía resultados.
 - Muestra puntaje.
 - Navega hacia JuegoGanadoController.
-

cambiarTema()

Descripción

Permite alternar entre modo claro y oscuro.

Objetivo

Mejorar experiencia visual.

5. JuegoGanadoController

Descripción General

Controlador encargado de mostrar el resumen final del juego.

Funciones principales:

- Mostrar resultados.
 - Mostrar puntaje final.
 - Continuar al siguiente nivel.
 - Reiniciar juego.
 - Alternar temas visuales.
-

Componentes FXML

Componente	Tipo	Función
tablaResultados	TableView	Tabla resumen
colEntrada	TableColumn	Entradas
colSalida	TableColumn	Resultados
lblTitulo	Label	Título final
lblPuntaje	Label	Puntaje obtenido
btnSiguienteNivel	Button	Avanzar nivel
btnTema	Button	Cambio tema
panelPuntajeFinal	VBox	Panel puntaje

Componente	Tipo	Función
lblPuntajeFinal	Label	Puntaje total
lblMensajePuntaje	Label	Mensaje motivacional
btnVerPuntaje	Button	Mostrar/Ocultar puntaje

Clase Interna

FilaResultado

Descripción

Representa cada resultado mostrado en la tabla.

Objetivo

Mostrar resumen visual de respuestas.

Métodos Principales

initialize()

Descripción

Inicializa la tabla de resultados.

Funcionalidad

- Configura columnas.
 - Personaliza encabezados.
 - Establece estilos.
-

setJuego(Juego juego)

Descripción

Recibe la partida finalizada.

Funcionalidad

- Recupera puntaje.
 - Muestra información.
 - Actualiza resumen.
-

cargarResultados()

Descripción

Carga todas las respuestas del jugador.

Objetivo

Mostrar resumen completo del nivel.

mostrarPuntajeFinal()

Descripción

Muestra u oculta el panel de puntaje.

Funcionalidad

- Alterna visibilidad.
 - Actualiza mensajes.
-

siguienteNivel()

Descripción

Permite avanzar al siguiente nivel.

Flujo

1. Incrementa nivel.
 2. Inicializa nueva partida.
 3. Navega hacia fase 1.
-

reiniciarJuego()

Descripción

Reinicia completamente la partida.

Objetivo

Permitir volver a empezar.

cambiarTema()

Descripción

Alterna entre modo claro y oscuro.

Objetivo

Mejorar experiencia de usuario.

DOCUMENTACIÓN DE SERVICIOS

ConexionBD

Descripción

Clase Singleton encargada de administrar la conexión a la base de datos.

Funciones principales

- Crear conexión.
- Inicializar tablas.
- Mantener una única instancia.
- Gestionar persistencia.

Métodos principales

getInstancia()

Retorna la única instancia de conexión.

getConexion()

Devuelve el objeto Connection.

inicializarEsquema()

Crea tablas necesarias si no existen.

DAOJugador

Descripción

Gestiona operaciones CRUD de jugadores.

Funciones

- Crear jugadores.
 - Buscar jugadores.
 - Actualizar puntajes.
 - Validar duplicados.
-

DAOPartida

Descripción

Gestiona persistencia de partidas.

Funciones

- Guardar partidas.
 - Consultar historial.
 - Registrar progreso.
-

DOCUMENTACIÓN DEL MODELO

Juego

Descripción

Clase principal del sistema.

Responsabilidades

- Controlar niveles.
- Gestionar puntaje.

- Administrar jugador.
 - Coordinar reglas.
-

Jugador

Descripción

Representa al usuario del sistema.

Atributos

- id
 - nombre
 - puntaje
-

Nivel

Descripción

Representa cada nivel del juego.

Funciones

- Definir dificultad.
 - Gestionar reglas.
 - Controlar progresión.
-

Regla

Descripción

Define la transformación matemática aplicada.

Función

Procesar entradas y generar salidas.

TablaIO

Descripción

Representa pares entrada/salida.

Objetivo

Mostrar comportamiento de las reglas.

CONCLUSIONES

El proyecto JUEGO MENTAL MATEMÁTICO implementa correctamente una arquitectura MVC utilizando JavaFX y Java. La separación entre modelo, vista, controladores y servicios facilita la escalabilidad, el mantenimiento y la comprensión del sistema.

Los controladores cumplen un papel fundamental al gestionar:

- Navegación entre ventanas.
- Validación de datos.
- Interacción con el usuario.
- Comunicación con el modelo.
- Persistencia de información.

El sistema incorpora además persistencia mediante base de datos, validaciones, gestión de puntajes, múltiples niveles y una interfaz gráfica moderna.

Finalmente, el proyecto demuestra la integración de conceptos fundamentales de ingeniería de software como:

- Programación orientada a objetos.
- Patrón MVC.
- Persistencia de datos.
- Interfaces gráficas.
- Arquitectura modular.
- Validaciones.
- Navegación dinámica.
- Gestión de eventos

